IT3280 – THỰC HÀNH KIẾN TRÚC MÁY TÍNH

BÁO CÁO THỰC HÀNH

BÀI 2: Tập lệnh, các lệnh cơ bản, các chỉ thị biên dịch

Họ và tên: Nguyễn Hồng Khôi

MSSV: 202416956

Assignment 1: Lệnh gán số nguyên nhỏ 12-bit

- Chương trình :

```
# Laboratory Exercise 2, Assignment 1
.text
    addi  s0,   zero,   0x512        # s0 = 0 + 0x512; I-type: chỉ có thể lưu
                                     # được hằng số có dấu 12 bits
```

```
add  s0,   x0,   zero          # s0 = 0 + 0 ; R-type: có thể sự dụng số
                               # hiệu thanh ghi thay cho tên thanh ghi
```

- **Ở mỗi lệnh quan sát cửa sổ Registers ta thấy sự thay đổi thanh ghi s0 và pc**

Khi chưa chạy lệnh :

s0	8	0x00000000
pc		0x00400000

Khi chạy lệnh thứ nhất: addi s0, zero, 0x512

s0	8	0x00000512
pc		0x00400004

Khi chạy lệnh thứ hai: add s0, x0, zero

s0	8	0x00000000
pc		0x00400008

Nhận xét:

+ Sau mỗi lệnh chạy ta thấy thanh ghi pc tăng thêm 4 để trỏ tới lệnh kế tiếp đúng như lý thuyết.

+ Thanh ghi s0 sau lệnh thứ nhất đã thay đổi từ 0x00000000 thành 0x00000512 do lệnh cộng addi đã cộng giá trị 0x512 với 0 rồi gán vào thanh ghi s0.

+ Sau lệnh thứ 2 thanh ghi s0 trở lại giá trị 0x00000000 do lệnh cộng add cộng x0 và zero (cũng là x0) kết quả là 0 được gán vào thanh ghi s0.

- **Trong cửa sổ Text Segment, so sánh mã máy của các lệnh trên với khuôn dạng lệnh**

Mã máy trong cửa sổ Text Segment:

Text Segment

Bkpt	Address	Code	Basic	Source
	0x00400000	0x51200413	addi x8,x0,0x00000512	3: addi s0, zero, 0x512 ...
	0x00400004	0x00000433	add x8,x0,x0	5: add s0, x0, zero ...

- **Sửa lại lệnh addi như bên dưới. Chuyện gì xảy ra sau đó. Hãy giải thích.**

  Lệnh cần sửa: addi s0, zero, 0x20232024

  Sau khi sửa lệnh addi và cho chạy thì chương trình báo lỗi

```
Error in D:\[Tài liệu học tập]\Thực hành KTMT\Lab2\Assignment_1.asm line 3 column 17: "0x20232024": operand is out of range
Assemble: operation completed with errors.
```

  Giải thích: Vì lệnh addi thuộc loại I-type nên giá trị imm chỉ có 12bit mà giá trị 0x20232024 vượt quá giá trị 12bit có thể biểu diễn được.

Assignment 2: Lệnh gán số 32-bit

- Chương trình:

```
# Laboratory Exercise 2, Assignment 2
# Load 0x20232024 to s0 register
.text
        lui s0, 0x20232          # s0 = 0x20232
addi s0, s0, 0x024          # s0 = s0 + 0x024
```

- **Quan sát cửa sổ Registers, theo dõi sự thay đổi của các thanh ghi s0, pc**

Khi chưa chạy lệnh:

s0	8	0x00000000
pc		0x00400000

Sau khi chạy lệnh thứ nhất: lui s0, 0x20232

s0	8	0x20232000
pc		0x00400004

Sau khi chạy lệnh thứ hai: addi s0, s0, 0x024

s0	8	0x20232024

pc		0x00400008

Nhận xét:

+ Thanh ghi pc tăng thêm 4 sau mỗi lần chạy lệnh để trỏ đến lệnh kế tiếp.

+ Thanh ghi s0 sau lệnh 1 chuyển từ 0x00000000 thành 0x20232000 do lệnh lui đã nạp 20 bit trọng số cao 0x20232 vào thanh ghi.

+ Thanh ghi s0 sau lệnh 2 chuyển từ 0x20232000 thành 0x20232024 do lệnh addi đã nạp 12 bit trọng số thấp 0x024 vào thanh ghi.

- **So sánh dữ liệu trong vùng Data Segment và mã máy trong Text Segment.**

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x00400000	0x20232437	0x02440413	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00400020	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00400040	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00400060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00400080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x004000a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x004000c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x004000e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00400100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

0x00400000 (.text) ☑ Hexadecimal Addresses ☑ Hexadecimal Values ☐ ASCII

Text Segment

Bkpt	Address	Code	Basic	Source
☐	0x00400000	0x20232437	lui x8,0x00020232	4: lui s0, 0x20232 # s0 = 0x20232
☐	0x00400004	0x02440413	addi x8,x8,0x00000024	5: addi s0, s0, 0x024 # s0 = s0 + 0x..

Nhận xét:

+ Dữ liệu trong vùng Data Segment giống mã máy trong Text Segment.

+ Trong Data Segment, các câu lệnh được để ở các địa chỉ 0x00400000 và 0x000400004, các câu lệnh được xếp cạnh nhau. Thanh ghi pc có thể đọc lệnh khi di chuyển 4 byte.

- **Tại sao lệnh addi s0, s0, 0x900 gây ra lỗi biên dịch còn lệnh addi s0, s0, 0xFFFFF900 hợp lệ ?**

Lệnh **addi s0, s0, 0x900** gây ra lỗi biên dịch, do giá trị 0x900 được mở rộng thành 32-bit theo kiểu zero-entend thành 0x00000900 tương đương giá trị 2304

ở hệ 10 nằm ngoài dải biểu diễn số nguyên có dấu 12-bit, không phù hợp với tham số immediate của lệnh addi.

Lệnh **addi s0, s0, 0xFFFFF900** hợp lệ, giá trị 0x900 được mở rộng thành 32bit theo kiểu sign-extend, khi chuyển sang hệ 10 vẫn mang giá trị -1792, là tham số hợp lệ cho lệnh addi.

- **Khi nạp một số 32-bit vào thanh ghi, nếu số 12-bit trong lệnh addi là số âm (bit thứ 11 bằng 1), thì cần mở rộng dấu thành 32 bit, và số 20-bit trong lệnh lui cần phải tăng lên 1. Giải thích tại sao?**

Giải thích:

+ Khi số 12 bit trong lệnh addi là số âm, cần mở rộng dấu thành 32 bit với kiểu sign-extend, sẽ có dạng 0xFFFFFabc. Nếu thực hiện lệnh addi để cộng s0 hiện tại được nạp 20 bit trong số cao bởi lui và 12 bit trọng số thấp đã được chuyển thành 32 bit sẽ khiến cho 20 bit trọng số cao bị giảm đi 1 đơn vị. Do đó cần tăng số 20 bit trong lệnh lui lên 1.

Khi thực hiện lệnh addi với 0xFEEDB000 cho ra kết quả là 0xFEEDA900

Chương trình:

.text

lui s0, 0xFEEDB # s0 = 0xFEEDB000 addi s0, s0, 0xFFFFF900 # s0 = s0 + 0xFFFFF900 Thanh ghi:

Name	Number	Value
zero	0	0x00000000
ra	1	0x00000000
sp	2	0x7fffeffc
gp	3	0x10008000
tp	4	0x00000000
t0	5	0x00000000
t1	6	0x00000000
t2	7	0x00000000
s0	8	0xfeeda900
s1	9	0x00000000
a0	10	0x00000000

Assignment 3: Lệnh gán (giả lệnh)

- **Biên dịch, quan sát và so sánh các lệnh ở cột Source và cột Basic trong cửa sổ Text Segment. Giải thích kết quả.**

Chương trình:

```
# Laboratory Exercise 2, Assignment 3

.text        li s0,

0x20232024        li s0,
0x20
```

Các lệnh trong cột Source và cột Basic:

Text Segment

Bkpt	Address	Code	Basic	Source
	0x00400000	0x20232437	lui x8,0x00020232	3: li s0, 0x20232024
	0x00400004	0x02440413	addi x8,x8,0x00000024	
	0x00400008	0x02000413	addi x8,x0,0x00000020	4: li s0, 0x20

Giải thích kết quả:

- Lệnh li không phải là một phần của bộ lệnh RISC-V thực tế. Nó là một lệnh giả (pseudo-instruction) được cung cấp để đơn giản hóa việc lập trình. Trình dịch hợp ngữ (assembler) sẽ chuyển đổi lệnh li thành các lệnh RISC-V thực tế, sử dụng lui và addi.

3.1 Dòng đầu tiên: li s0, 0x20232024: nạp một giá trị 32-bit lớn (0x20232024) vào thanh ghi s0.

- Vì li cần xử lý các số lớn (lớn hơn 12 bit), nó sử dụng hai lệnh:

  lui: Nạp 20 bit cao của giá trị tức thời. addi:

  Cộng thêm 12 bit thấp.

  li s0, 0x20232024 được dịch thành hai lệnh:

  lui s0, 0x20232 và addi s0, s0, 0x024.

3.2 Dòng thứ hai: li s0, 0x20: nạp giá trị tức thời nhỏ hơn 0x20 vào thanh ghi s0.

- Vì giá trị này có thể nằm trong phạm vi 12 bit của trường tức thời trong lệnh addi, nên không cần sử dụng lui. Trình dịch hợp ngữ chỉ sử dụng một lệnh: addi s0, zero, 0x20 (cộng trực tiếp 0x20 vào s0).

- li s0, 0x20 được dịch thành một lệnh duy nhất: addi s0, zero, 0x20.

Assignment 4: Tính biểu thức 2x + y = ?

- **Quan sát cửa sổ Registers, theo dõi sự thay đổi của các thanh ghi.**

Chương trình:

```
# Laboratory Exercise 2, Assignment 4

.text

# Assign X, into t1, t2 register
        addi t1, zero, 5       # X = t1 = ?
 addi t2, zero, -1  # Y = t2 = ?
```

Expression Z = 2X + Y add s0, t1, t1 # s0 = t1 + t1 = X + X = 2X add s0, s0, t2 # s0 = s0 + t2 = 2X + Y

Khi chưa chạy lệnh:

t0	5	0x00000000
t1	6	0x00000000
t2	7	0x00000000
s0	8	0x00000000

Khi chạy lệnh 1:

t0	5	0x00000000
t1	6	0x00000005
t2	7	0x00000000
s0	8	0x00000000

Khi chạy lệnh 2:

t0	5	0x00000000
t1	6	0x00000005
t2	7	0xffffffff
s0	8	0x00000000

Khi chạy lệnh 3:

t0	5	0x00000000
t1	6	0x00000005
t2	7	0xffffffff
s0	8	0x0000000a
s1	9	0x00000000

Khi chạy lệnh 4:

t0	5	0x00000000
t1	6	0x00000005
t2	7	0xffffffff
s0	8	0x00000009

+ Nhờ các lệnh addi, giá trị của t1 và t2 lần lượt được gán bằng 0x00000005 và 0xffffffff là 5 và -1.

+ Lệnh add s0, t1, t1 cộng t1 và t1 sau đó gán vào thanh ghi s0 được giá trị 0x0000000a tương ứng với 10.

+ Lệnh add s0, s0, t2 cộng giá trị s0 hiện tại và t2 sau đó gán vào thanh ghi s0 được giá trị là 0x00000009 tương ứng với 9 (10+(-1) = 9).

Kết luận: Sau khi chương trình chạy xong, kết quả của phép tính 2x+y được tính đúng

- **Trong cửa sổ Text Segment, quan sát mã máy của các các lệnh. Từ đó kiểm nghiệm mã máy với khuôn mẫu của lệnh addi (I-type) và lệnh add (R-type).**

Text Segment

Bkpt	Address	Code	Basic	Source
	0x00400000	0x00500313	addi x6,x0,5	4: addi t1, zero, 5 # X = t1 = ?
	0x00400004	0xfff00393	addi x7,x0,0xfffffffff	5: addi t2, zero, -1 # Y = t2 = ?
	0x00400008	0x00630433	add x8,x6,x6	8: add s0, t1, t1 # s0 = t1 + t1 ..
	0x0040000c	0x00740433	add x8,x8,x7	9: add s0, s0, t2 # s0 = s0 + t2 ..

Assignment 5: Phép nhân

Chương trình:

```
# Laboratory Exercise 2, Assignment 5

.text

# Assign X, Y into t1, t2 register  addi t1,
zero, 4   # X = t1 =? addi t2, zero, 5
 # Y = t2 =? # Expression Z = X * Y   mul s1,
t1, t2            # s1 chứa 32 bit thấp
```

- **Quan sát sự thay đổi của các thanh ghi:**

Trước khi chạy lệnh:

t0	5	0x00000000
t1	6	0x00000000
t2	7	0x00000000
s0	8	0x00000000
s1	9	0x00000000

Khi chạy lệnh 1:

t0	5	0x00000000
t1	6	0x00000004
t2	7	0x00000000
s0	8	0x00000000
s1	9	0x00000000

Khi chạy lệnh 2:

t0	5	0x00000000
t1	6	0x00000004
t2	7	0x00000005
s0	8	0x00000000
s1	9	0x00000000

Khi chạy lệnh 3:

t0	5	0x00000000
t1	6	0x00000004
t2	7	0x00000005
s0	8	0x00000000
s1	9	0x00000014

Nhận xét:

+ Các lệnh addi gán cho các biến t1, t2 các giá trị 0x00000004 (4) và 0x00000005 (5) + Lệnh mul thực hiện phép nhân với các nhân tử là t1 và t2 kết quả cho ra là 20 được gán vào thanh ghi s1 nên thanh s1 biểu thị 0x0000001

❾ Kết quả cho ra là đúng.

- **Tìm hiểu lệnh chia, lập trình ví dụ minh họa, giải thích luồng hoạt động cũng như kết quả của các thanh ghi.**

Lệnh chia trong RISC-V là div và divu lần lượt là phép chia lấy thương giữa 2 số nguyên có dấu và 2 số nguyên không dấu

```
div t1,t2,t3        Division: set t1 to the result of t2/t3
divu t1,t2,t3       Division: set t1 to the result of t2/t3 using unsigned division
```

Cách hoạt động: lệnh div gán vào thanh ghi t1 kết quả của phép chia t2 cho t3 lấy thương. Lệnh divu tương tự nhưng các số hạng không có dấu

Ví dụ minh hoạ :

.text addi t1, zero, -10
addi t2, zero, 3 div s0,
t1, t2 Theo dõi kết quả:

t1	6	0xfffffff6
t2	7	0x00000000
s0	8	0x00000000
s1	9	0x00000000

t1	6	0xfffffff6
t2	7	0x00000003
s0	8	0x00000000

t1	6	0xfffffff6
t2	7	0x00000003
s0	8	0xfffffffd

Kết quả của phép chia nằm trong thanh ghi s1 là 0xfffffffd. Phép chia -10 cho 3 lấy phần thương được kết quả là -3.

❾ Lệnh hoạt động đúng

Assignment 6: Tạo biến và truy cập biến Chương trình:

```
# Laboratory Exercise 2, Assignment 6
.data  # Khởi tạo biến (declare memory)
        X: .word 5 # Biến X, kiểu word (4 bytes), giá trị khởi tạo = 5
        Y: .word -1 # Biến Y, kiểu word (4 bytes), giá trị khởi tạo = -1
        Z: .word 0 # Biến Z, kiểu word (4 bytes), giá trị khởi tạo = 0

.text  # Khởi tạo lệnh (declare instruction)
      # Nạp giá trị X và Y vào các thanh ghi          la t5, X #
Lấy địa chỉ của X trong vùng nhớ chứa dữ liệu       la t6, Y #
Lấy địa chỉ của Y   lw t1, 0(t5) # t1 = X        lw t2, 0(t6) # t2 =
Y
```

Tính biểu thức Z = 2X + Y với các thanh ghi add s0, t1, t1 add s0, s0, t2

Lưu kết quả từ thanh ghi vào bộ nhớ la t4, Z # Lấy địa chỉ của Z sw s0, 0(t4) # Lưu giá trị của Z từ thanh ghi vào bộ nhớ

Text Segment

Bkpt	Address	Code	Basic	Source
	0x00400000	0x0fc10f17	auipc x30,0x0000fc10	9: la t5, X # L?y ??a ch? c?a X ..
	0x00400004	0x000f0f13	addi x30,x30,0	
	0x00400008	0x0fc10f97	auipc x31,0x0000fc10	10: la t6, Y # L?y ??a ch? c?a Y
	0x0040000c	0xffcf8f93	addi x31,x31,0xfffffffc	
	0x00400010	0x000f2303	lw x6,0(x30)	11: lw t1, 0(t5) # t1 = X
	0x00400014	0x000fa383	lw x7,0(x31)	12: lw t2, 0(t6) # t2 = Y
	0x00400018	0x00630433	add x8,x6,x6	15: add s0, t1, t1
	0x0040001c	0x00740433	add x8,x8,x7	16: add s0, s0, t2
	0x00400020	0x0fc10e97	auipc x29,0x0000fc10	19: la t4, Z # L?y ??a ch? c?a Z
	0x00400024	0xfe8e8e93	addi x29,x29,0xffffffe8	
	0x00400028	0x008ea023	sw x8,0(x29)	20: sw s0, 0(t4) # L?u giá tr? c?..

- **Lệnh la (load address) được biên dịch như thế nào?**

```
la t1,label                Load Address : Set t1 to label's address
```

Lệnh la (Load Address) là một **lệnh giả** (pseudo-instruction), dùng để tải địa chỉ của một nhãn (label) vào thanh ghi. Khi biên dịch nó được chuyển thành 2 lệnh chính thống là auipc và addi

```
auipc t1,100000          Add upper immediate to pc: set t1 to (pc plus an upper 20-bit immediate)
```

Lệnh auipc (Add Upper Immediate to PC) được dùng để tính địa chỉ tương đối với bộ đếm chương trình (PC).

- Nó tải giá trị tức thời 20 bit vào 20 bit cao của thanh ghi đích (rd) và cộng với giá trị của PC.
- Công thức: rd = PC + (imm << 12)

Chi tiết hoạt động:

- Lệnh la số 1 hoạt động như sau:

+ Lệnh auipc sẽ lấy giá trị thanh ghi pc hiện tại là 0x0040000 cộng với 0x0fc10000 (0x0000fc10 sau khi mở rộng 12 bit bên phải) kết quả cho ra là 0x10010000 và gán vào thanh ghi x30.

+ Lệnh addi cộng thêm giá trị của x30 với 0 kết quả ra được địa chỉ của biến X là 0x10010000

+ Theo dõi thanh ghi t5 trước và sau khi chạy lệnh **la t5, X**

t5	30	0x00000000
t6	31	0x00000000
pc		0x00400000

t5	30	0x10010000
t6	31	0x00000000
pc		0x00400004

Thanh ghi t5 thay đổi thành 0x10010000, thanh ghi đã được chép địa chỉ của X từ bộ nhớ.

- Lệnh la số 2 hoạt động như sau:

+ Lệnh auipc cũng hoạt động như trên, lấy giá trị thanh pc hiện tại 0x00400008 cộng với 0x0fc10000 kết quả là gán 0x10010008 vào thanh ghi x31.

+ Lệnh addi cộng thêm giá trị x30 với 0xfffffffc kết quả cho ra 0x10010004 là địa chỉ của biến Y.

+ Theo dõi thanh ghi t6 trước và sau khi chạy câu lệnh **la t6, Y**

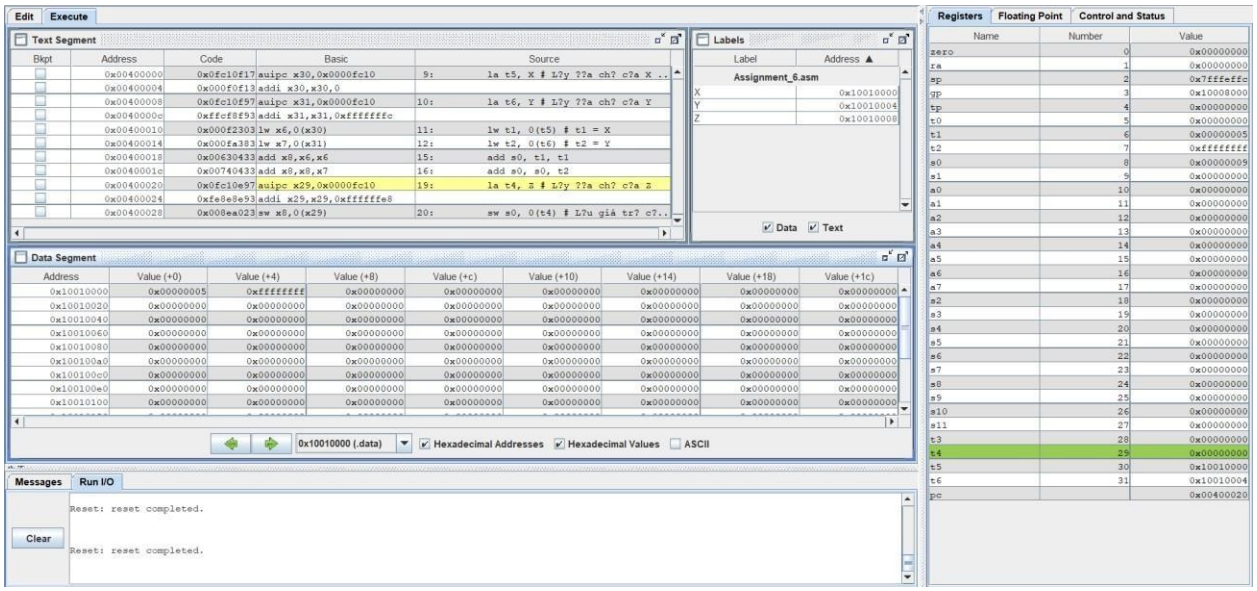
t5	30	0x10010000
t6	31	0x00000000
pc		0x00400008

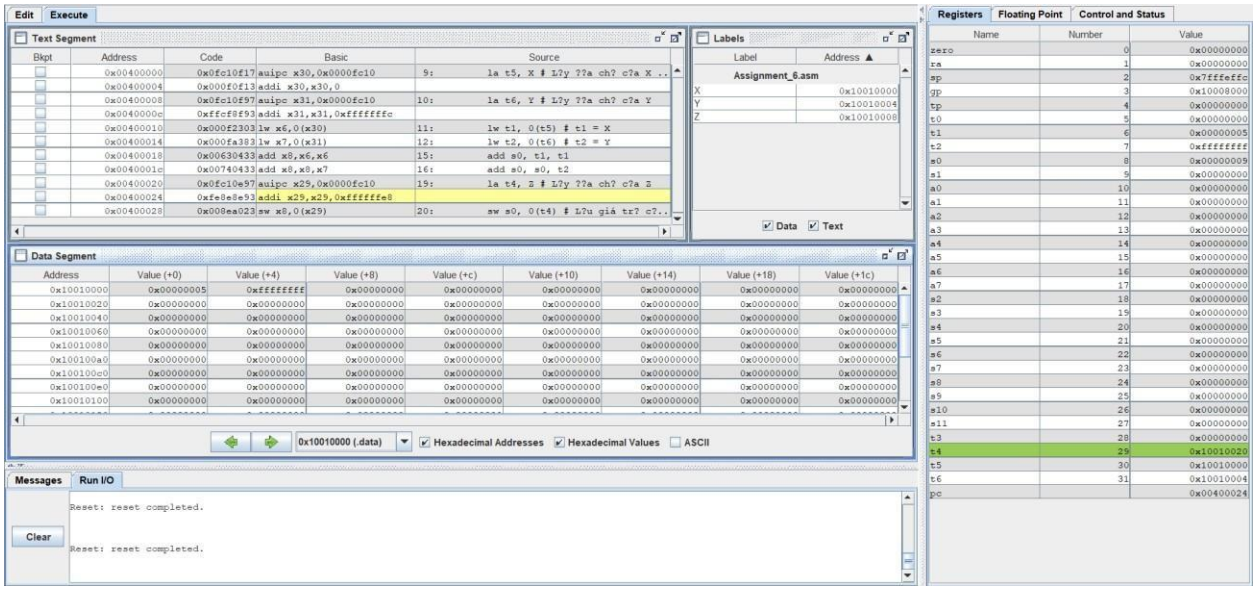
t5	30	0x10010000
t6	31	0x10010004
pc		0x00400010

Thanh ghi t6 đang chứa địa chỉ của biến Y

- Tương tự với lệnh la số 3, địa chỉ của pc là 0x00400020 kết quả cuối cùng là 0x10010008 là địa chỉ của biến Z.

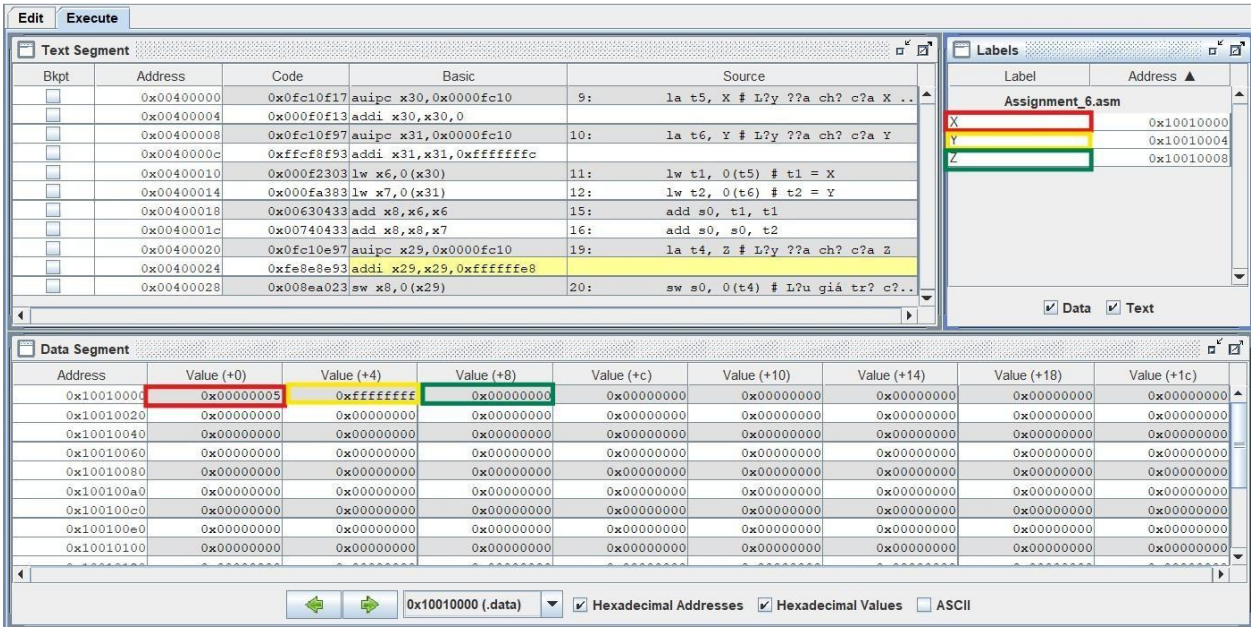
+ Theo dõi thanh ghi t4 trước và sau khi chạy câu lệnh **la t4, Z**





Thanh ghi t4 đang chứa địa chỉ của biến Z.

Trong cửa sổ Labels, xem địa chỉ của biến X, Y, Z được lưu trong bộ nhớ.



Các giá trị các biến X, Y, Z trong bộ nhớ lần lượt hoàn toàn trùng khớp với giá trị khởi tạo trong mã nguồn

- **Xác định vai trò của lệnh lw.**

```
lw t1, -100(t2)          Set t1 to contents of effective memory word address
```

+ Lệnh **lw rd, offset(rs1)** với rd là thanh ghi đích, offset là giá trị số byte từ địa chỉ của thanh ghi rs1 đến ví trí cần lấy dữ liệu. Lệnh này sẽ lưu trữ, nạp dữ liệu lấy được vào thanh ghi rd.

+ Với 2 câu lệnh đề bài đã cho đều có offset bằng 0, giá trị được nạp vào rd chính là giá trị của thanh ghi rs1.

Trạng thái thanh ghi trước và sau khi chạy các câu lệnh lw (load word)

t0	5	0x00000000
t1	6	0x00000000
t2	7	0x00000000
s0	8	0x00000000

t0	5	0x00000000
t1	6	0x00000005
t2	7	0x00000000
s0	8	0x00000000

t1	6	0x00000005
t2	7	0xffffffff
s0	8	0x00000000
s1	9	0x00000000

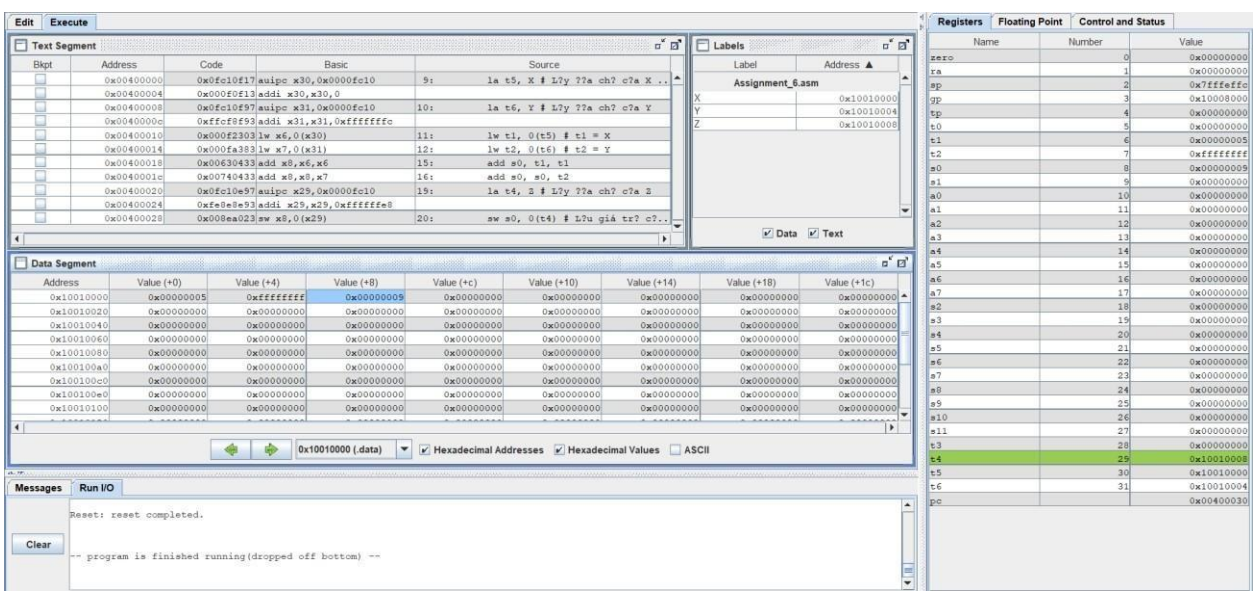
Vai trò của lệnh lw là nạp dữ liệu từ bộ nhớ vào thanh ghi.

- **Xác định vai trò của lệnh sw.**

```
sw t1, -100(t2)          Store word : Store contents of t1 into effective memory word address
```

+ Lệnh **sw rs2, offset(rs1)** có rs1 là thanh ghi nguồn, nơi chứa dữ liệu word cần ghi vào bộ nhớ, rs2 là thanh ghi chứa địa chỉ cơ sở, offset là hằng số dịch chuyển địa chỉ.

+ Câu lệnh đề bài cho có offset là 0, dữ liệu được ghi vào chính bộ nhớ mà địa chỉ bộ nhớ là giá trị của rs1



Với câu lệnh của đề bài cho, sau khi chạy lệnh sw, giá trị của biến s0 sẽ được ghi vào địa chỉ cơ sở được ghi trên thanh ghi t4 tức là 0x1001008.

=> Lệnh sw có vai trò lưu dữ liệu từ thanh ghi vào bộ nhớ

- **Tìm hiểu thêm các lệnh lb, sb**

```
lb t1, -100(t2)          Set t1 to sign-extended 8-bit value from effective memory byte address
sb t1, -100(t2)          Store byte : Store the low-order 8 bits of t1 into the effective memory byte address
```

+ Nếu lệnh lw (load word) và sw(store word) làm việc với dữ liệu dạng word (4byte) thì lb và sb có cách hoạt động tương tự nhưng làm việc với dữ liệu dạng byte (1byte).

<u>Tổng kết Assignment 6:</u> Lệnh la, lw, lb nạp giá trị của biến từ bộ nhớ vào thanh ghi. Sau khi xử lý dữ liệu. Lưu kết quả từ thanh ghi trở lại bộ nhớ của biến bằng các lệnh sw,